

## 第5节 树

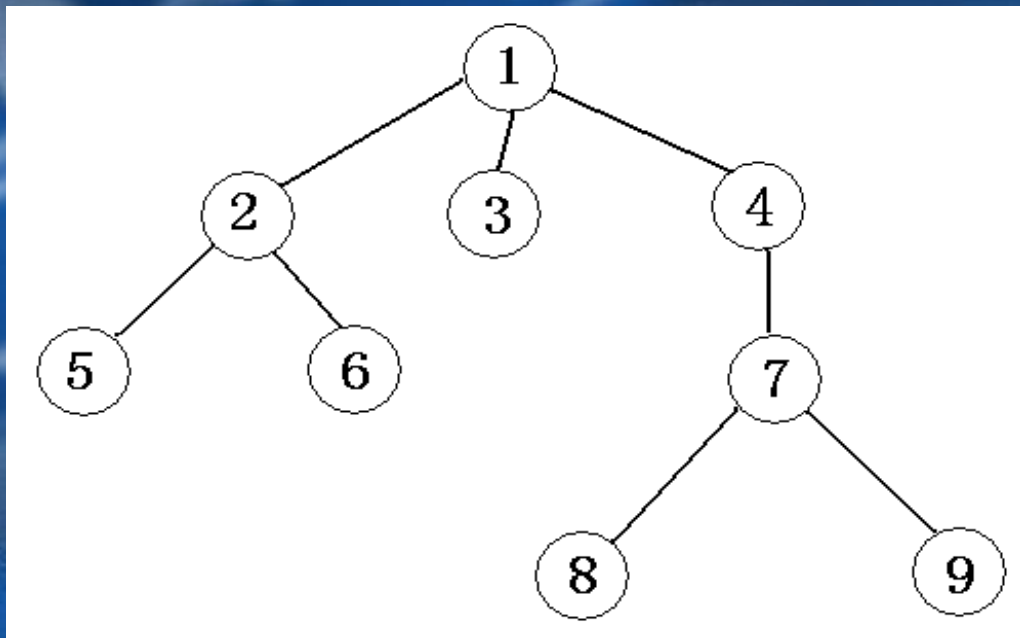
前两章学习的栈和队列属于线性结构。在这种结构中，数据元素的逻辑位置之间呈线性关系，每一个数据元素通常只有一个前件（除第一个元素外）和一个后件（除最后一个元素外）。在实际生活中，可以用线性结构描述数据元素之间逻辑关系的问题是很广泛的，但也有很多问题不能依靠线性结构来解决，例如家谱、行政组织机构等都是非线性的数据结构。其中树就是一种非线性的数据结构。

# 一、树的定义

一棵树是由 $n$  ( $n > 0$ ) 个元素组成的有限集合，其中：

- (1) 每个元素称为结点 (node)；
- (2) 有一个特定的结点，称为根结点或树根 (root)；
- (3) 除根结点外，其余结点能分成 $m$  ( $m \geq 0$ ) 个互不相交的有限集合  $T_0, T_1, T_2, \dots, T_{m-1}$ 。其中的每个子集又都是一棵树，这些集合称为这棵树的子树。

如下图是一棵典型的树：



## 二、树的基本概念

- 树是递归定义的；
- 一棵树中至少有1个结点。这个结点就是根结点，它没有前驱，其余每个结点都有唯一的一个前驱结点。每个结点可以有0或多个后继结点。因此树虽然是非线性结构，但也是有序结构。至于前驱后继结点是哪个，还要看树的遍历方法，我们将在后面讨论；
- 一个结点的子树个数，称为这个结点的度（degree，结点1的度为3，结点3的度为0）；度为0的结点称为叶结点（树叶leaf，如结点3、5、6、8、9）；度不为0的结点称为分支结点（如结点1、2、4、7）；根以外的分支结点又称为内部结点（结点2、4、7）；树中各结点的度的最大值称为这棵树的度（这棵树的度为3）。
- 在用图形表示的树型结构中，对两个用线段（称为树枝）连接的相关联的结点，称上端结点为下端结点的父结点，称下端结点为上端结点的子结点。称同一个父结点的多个子结点为兄弟结点。如结点1是结点2、3、4的父结点，结点2、3、4是结点1的子结点，它们又是兄弟结点，同时结点2又是结点5、6的父结点。称从根结点到某个子结点所经过的所有结点为这个子结点的祖先。如结点1、4、7是结点8

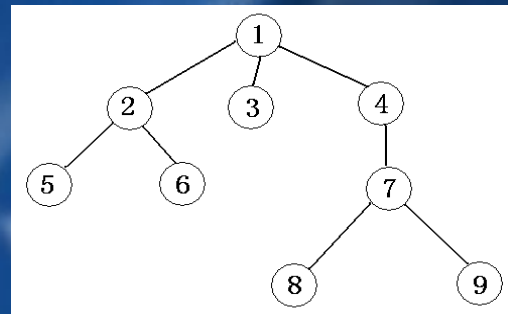


的祖先。称以某个结点为根的子树中的任一结点都是该结点的子孙。如结点7、8、9都是结点4的子孙。

- E. 定义一棵树的根结点的层次(level)为1, 其它结点的层次等于它的父结点层次加1。如结点2、3、4的层次为2, 结点5、6、7的层次为3, 结点8、9的层次为4。一棵树中所有的结点的层次的最大值称为树的深度(depth)。如这棵树的深度为4。
- F. 对于树中任意两个不同的结点, 如果从一个结点出发, 自上而下沿着树中连着结点的线段能到达另一结点, 称它们之间存在着一条路径。可用路径所经过的结点序列表示路径, 路径的长度等于路径上的结点个数减1。如上图中, 结点1和结点8之间存在着一条路径, 并可用(1、4、7、8)表示这条路径, 该条路径的长度为3。注意, 不同子树上的结点之间不存在路径, 从根结点出发, 到树中的其余结点一定存在着一条路径。
- G. 森林(forest)是 $m(m \geq 0)$ 棵互不相交的树的集合。

### 三、树的遍历

在应用树结构解决问题时，往往要求按照某种次序获得树中全部结点的信息，这种操作叫作树的遍历。遍历的方法有多种，常用的有：



A、先序（根）遍历：先访问根结点，  
再从左到右按照先序思想遍历各棵子树。

如上图先序遍历的结果为：125634789；

B、后序（根）遍历：先从左到右遍历各棵子树，再访问根结点。如  
上图后序遍历的结果为：562389741；

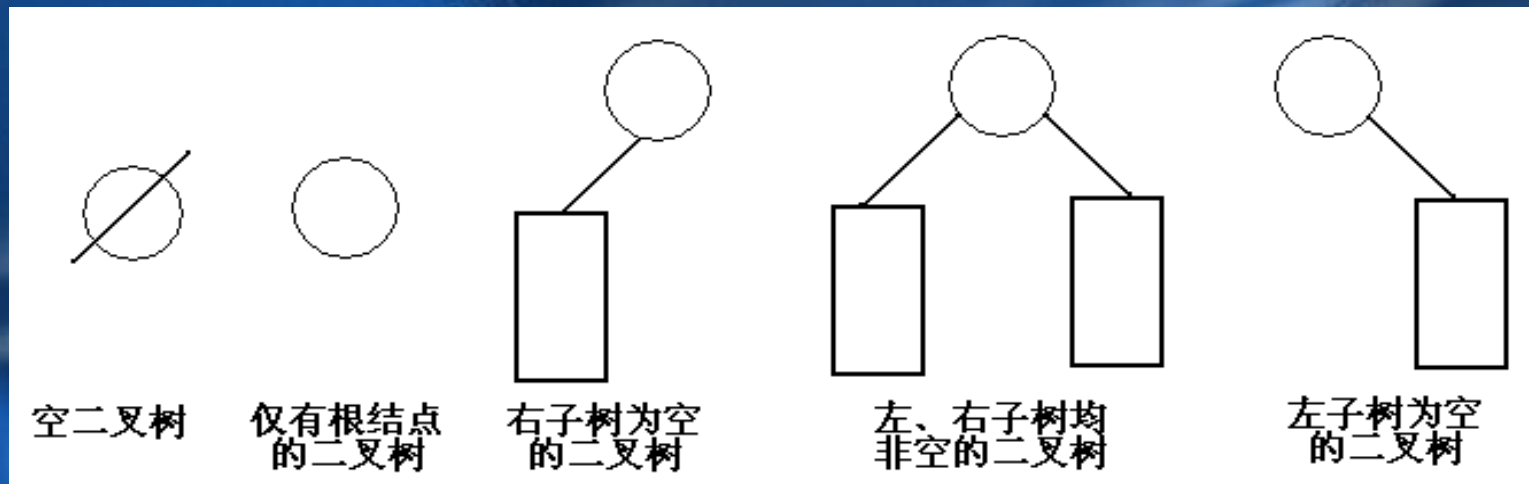
C、层次遍历：按层次从小到大逐个访问，同一层次按照从左到右的  
次序。

如上图层次遍历的结果为：123456789；

D、叶结点遍历：有时把所有的数据信息都存放在叶结点中，而其余  
结点都是用来表示数据之间的某种分支或层次关系，这种情况就 用这  
种方法。如上图按照这个思想访问的结果为：56389；

## 四、二叉树基本概念

二叉树 (binary tree, 简写成BT) 是一种特殊的树型结构, 它的度数为2的树。即二叉树的每个结点最多有两个子结点。每个结点的子结点分别称为左孩子、右孩子, 它的两棵子树分别称为左子树、右子树。二叉树有5中基本形态:



前面引入的树的术语也基本适用于二叉树, 但二叉树与树也有很多不同, 如: 首先二叉树的每个结点至多只能有两个结点, 二叉树可以为空, 二叉树一定是有序的, 通过它的左、右子树关系体现出来。



## 五、二叉树的性质

- **【性质1】** 在二叉树的第 $i$ 层上最多有 $2^{i-1}$ 个结点 ( $i \geq 1$ )。

证明：很简单，用归纳法：当 $i=1$ 时， $2^{i-1}=1$ 显然成立；现在假设第 $i-1$ 层时命题成立，即第 $i-1$ 层上最多有 $2^{i-2}$ 个结点。由于二叉树的每个结点的度最多为2，故在第 $i$ 层上的最大结点数为第 $i-1$ 层的2倍，即

$$2 * 2^{i-2} = 2^{i-1}。$$

- **【性质2】** 深度为 $k$ 的二叉树至多有 $2^k - 1$ 个结点 ( $k \geq 1$ )。

证明：在具有相同深度的二叉树中，仅当每一层都含有最大结点数时，其树中结点数最多。因此利用性质1可得，深度为 $k$ 的二叉树的结点数至多为：

$$2^0 + 2^1 + \dots + 2^{k-1} = 2^k - 1$$

故命题正确。

- **【特别】** 一棵深度为 $k$ 且有 $2^k - 1$ 个结点的二叉树称为满二叉树。如下图A为深度为4的满二叉树，这种树的特点是每层上的结点数都是最大结点数。

可以对满二叉树的结点进行连续编号，约定编号从根结点起，自上而下，从左到右，由此引出完全二叉树的定义，深度为 $k$ ，有 $n$ 个结点的二叉树当且仅当其每一个结点都与深度为 $k$ 的满二叉树中编号从1到 $n$ 的结点一一对应时，称为完全二叉树。

下图B就是一个深度为4，结点数为12的完全二叉树。它有如下特征：叶结点只可能在层次最大的两层上出现；对任一结点，若其右分支下的子孙的最大层次为 $m$ ，则在其左分支下的子孙的最大层次必为 $m$ 或 $m+1$ 。下图C、D不是完全二叉树，请大家思考为什么？



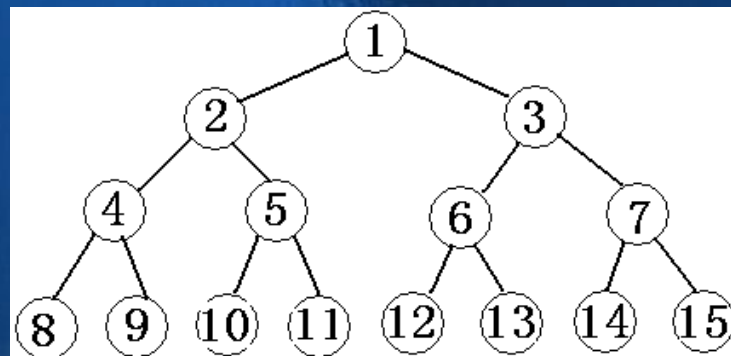


图 A

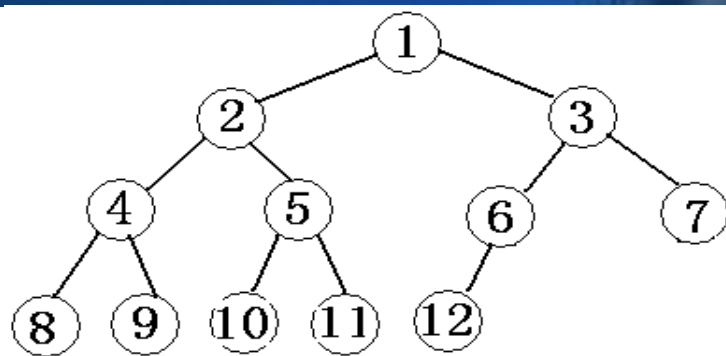


图 B

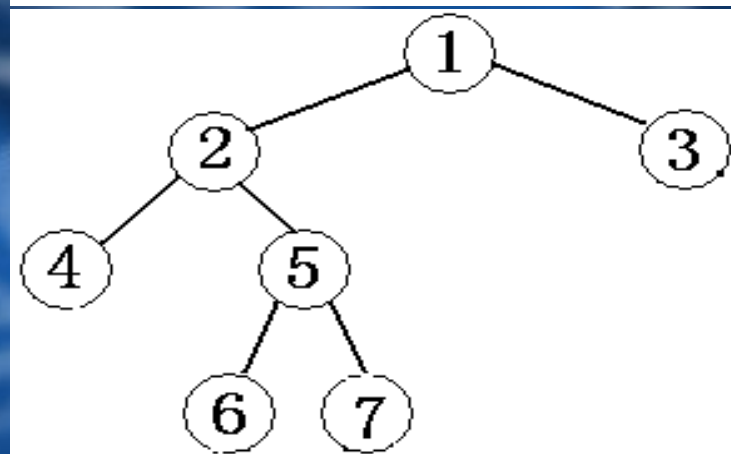


图 C

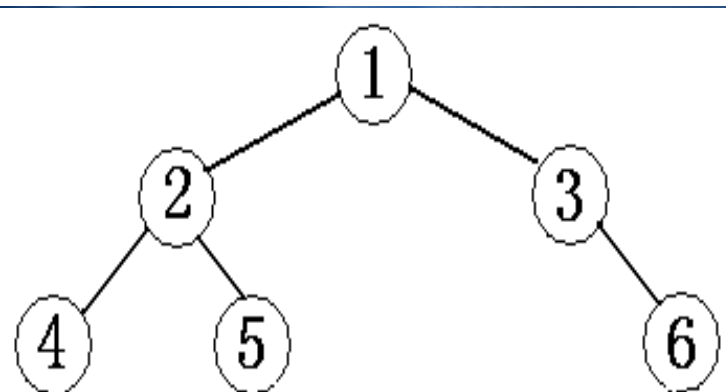


图 D

- **【性质3】** 对任意一棵二叉树，如果其叶结点数为 $n_0$ ，度为2的结点数为 $n_2$ ，则一定满足： $n_0=n_2+1$ 。

证明：因为二叉树中所有结点的度数均不大于2，所以结点总数(记为 $n$ )应等于0度结点数 $n_0$ 、1度结点 $n_1$ 和2度结点数 $n_2$ 之和：

$$n=n_0+n_1+n_2 \quad \cdots \cdots (\text{式子1})$$

另一方面，1度结点有一个孩子，2度结点有两个孩子，故二叉树中孩子结点总数是：

$$n_1+2n_2$$

树中只有根结点不是任何结点的孩子，故二叉树中的结点总数又可表示为：

$$n=n_1+2n_2+1 \quad \cdots \cdots (\text{式子2})$$

由式子1和式子2得到：

$$n_0=n_2+1$$

**【性质4】** 具有 $n$ 个结点的完全二叉树的深度为 $\text{floor}(\log_2 n) + 1$

证明：假设深度为 $k$ ，则根据完全二叉树的定义，前面 $k-1$ 层一定是满的，所以 $n > 2^{k-1} - 1$ 。但 $n$ 又要满足 $n \leq 2^k - 1$ 。所以， $2^{k-1} - 1 < n \leq 2^k - 1$ 。变换一下为 $2^{k-1} \leq n < 2^k$ 。

以2为底取对数得到： $k-1 \leq \log_2 n < k$ 。而 $k$ 是整数，所以 $k = \text{floor}(\log_2 n) + 1$ 。

**【性质5】** 对于一棵 $n$ 个结点的完全二叉树，对任一个结点(编号为 $i$ )，有：

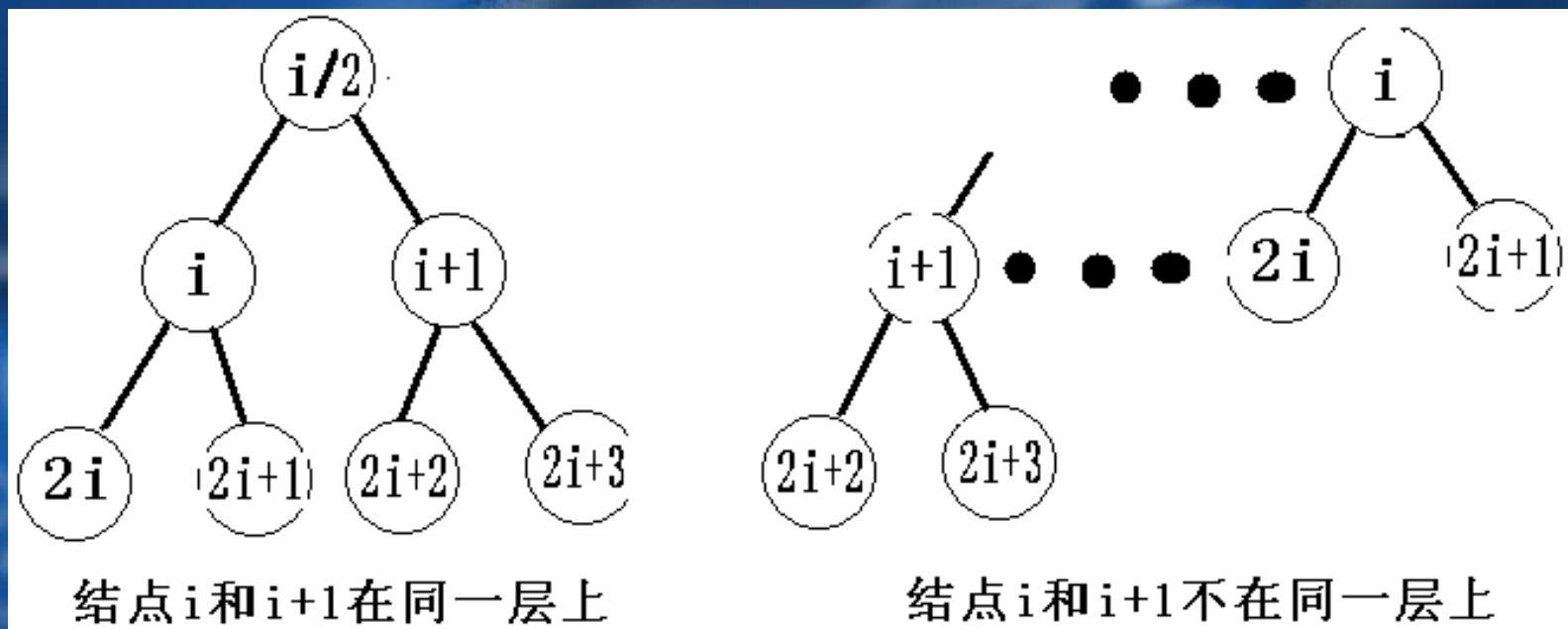
①如果 $i=1$ ，则结点 $i$ 为根，无父结点；如果 $i>1$ ，则其父结点编号为 $i/2$ 。

如果 $2*i > n$ ，则结点 $i$ 无左孩子（当然也无右孩子，为什么？即结点 $i$ 为叶结点）；否则左孩子编号为 $2*i$ 。

②如果 $2*i+1 > n$ ，则结点 $i$ 无右孩子；否则右孩子编号为 $2*i+1$ 。



证明：略，我们只要验证一下即可。总结如图：



## 六、遍历二叉树

在二叉树的应用中，常常要求在树中查找具有某种特征的结点，或者对全部结点逐一进行某种处理。这就是二叉树的遍历问题。所谓二叉树的遍历是指按一定的规律和次序访问树中的各个结点，而且每个结点仅被访问一次。“访问”的含义很广，可以是对结点作各种处理，如输出结点的信息等。遍历一般按照从左到右的顺序，共有3种遍历方法，先（根）序遍历，中（根）序遍历，后（根）序遍历。

- (一)先序遍历的操作定义如下：

若二叉树为空，则空操作，否则

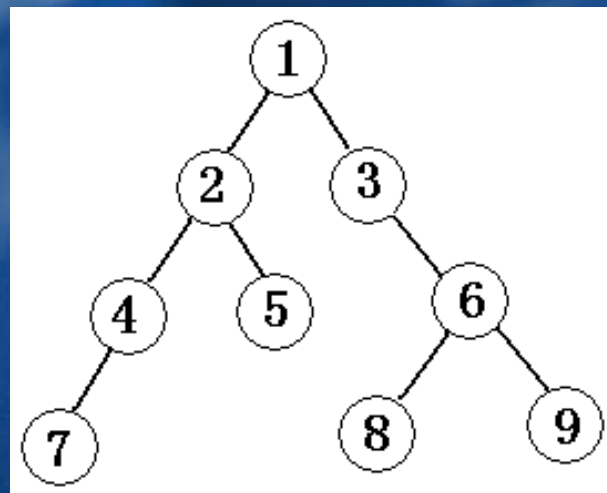
①访问根结点

②先序遍历左子树

③先序遍历右子树

```
void preorder(tree bt) //先序遍历根结点为bt的二叉树的递归算法
{
    if(bt)
    {
        cout << bt->data;
        preorder(bt->lchild);
        preorder(bt->rchild);
    }
}
```

先序遍历此图结果为：124753689



- (二)中序遍历的操作定义如下:

若二叉树为空, 则空操作, 否则

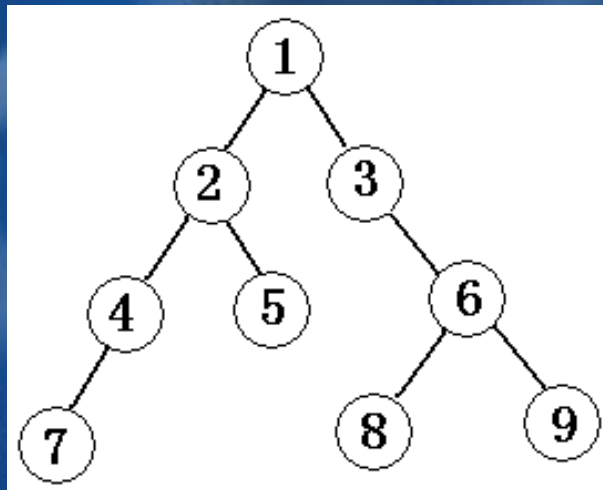
①中序遍历左子树

②访问根结点

③中序遍历右子树

```
void inorder(tree bt) //中序遍历根结点为bt的二叉树的递归算法
{
    if(bt)
    {
        inorder(bt->lchild);
        cout << bt->data;
        inorder(bt->rchild);
    }
}
```

中序遍历此图结果为: 742513869



- (三)后序遍历的操作定义如下:

若二叉树为空，则空操作，否则

①后序遍历左子树

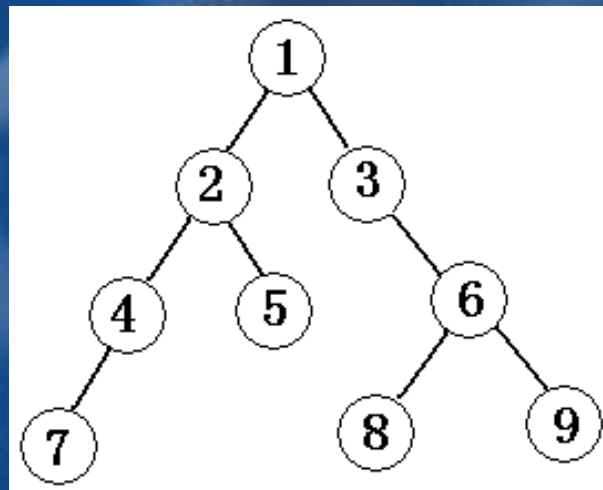
②后序遍历右子树

③访问根结点

void postorder(tree bt) //后序遍历根结点为bt的二叉树的递归算法

```
{  
  if(bt)  
  {  
    postorder(bt->lchild);  
    postorder(bt->rchild);  
    cout << bt->data;  
  }  
}
```

后序遍历此图结果为：745289631



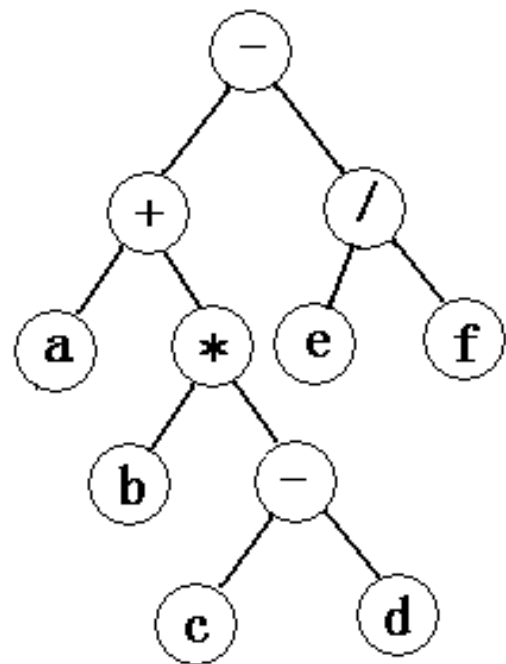


关于前面讲的表达式树，我们可以分别用先序、中序、后序的遍历方法得出完全不同的遍历结果，如对于下图中遍历结果如下，它们正好对应着表达式的3种表示方法。

$-+a*b-cd/ef$  (前缀表示、波兰式)

$a+b*c-d-e/f$  (中缀表示)

$abcd-*+ef/-$  (后缀表示、逆波兰式)



表达式  $(a+b*(c-d)-e/f)$  的二叉树

**结论：**已知前序序列和中序序列可以确定出二叉树；  
已知中序序列和后序序列也可以确定出二叉树；  
但，已知前序序列和后序序列却不可以确定出二叉树；为什么？

**例题3：**有二叉树中序序列为A B C E F G H D，后序序列为：A B F H G E D C。请画出此二叉树，并求前序序列。

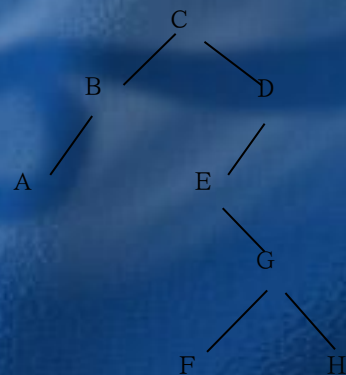
**解答：**根据后序序列知根节点为C

因此左子树：中序序列为A B  
后序序列为A B

右子树：中序序列为E F G H D  
后序序列为F H G E D

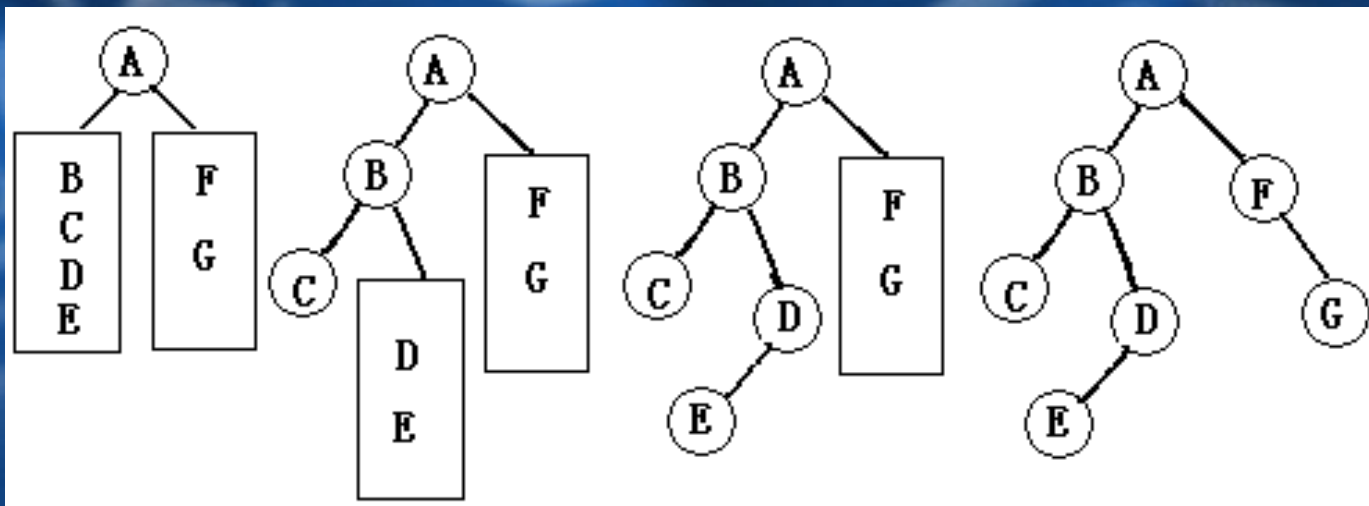
依次推得该二叉树的结构图。（如右图）

前序序列为：C B A D E G F H。



结论：已知先序序列和中序序列可以确定出二叉树；  
已知中序序列和后序序列也可以确定出二叉树；  
但，已知先序序列和后序序列却不可以确定出二叉树；为什么？

例题4：已知结点的先序序列为ABCDEFGG，中序序列为CBEDAFGG。构造出二叉树。过程见下图：



# 【课堂练习】



1、【NOIP2001提高组】一棵二叉树的高度为 $h$ ，所有结点的度为0，或为2，则此树最少有（ ）个结点

A.  $2^h-1$

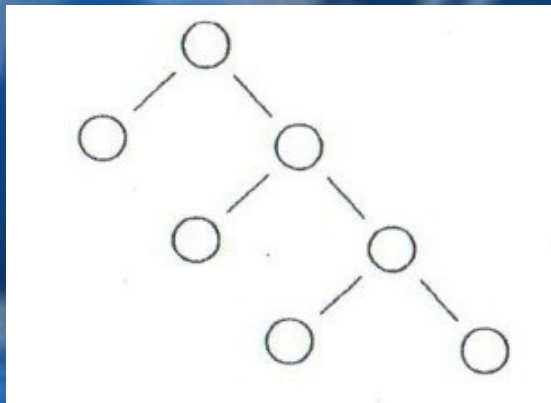
B.  $2h-1$

C.  $2h+1$

D.  $h+1$

【答案】B

【分析】符合题意最小节点的二叉树是如下形状：



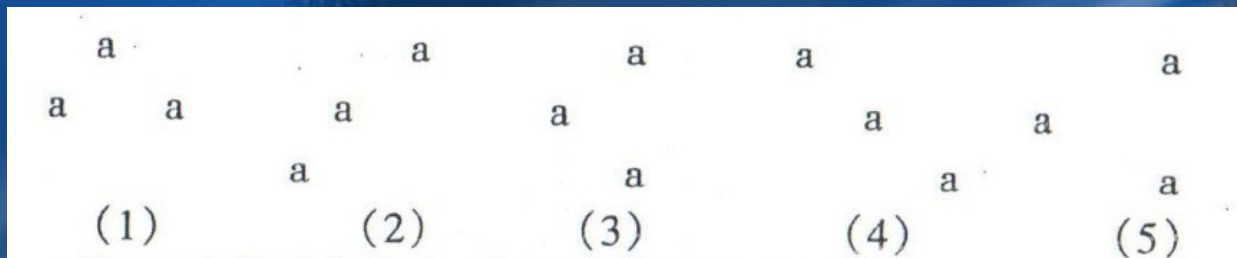
除根节点所在层节点数为1外，每层均为2个节点，因此节点数为 $2h-1$ 。

2、【NOIP2002提高组】按照二叉数的定义，具有3个结点的二叉树有（ ）种。

A. 3      B. 4      C. 5      D. 6

【答案】C

【分析】通过画图可知只有以下五种：

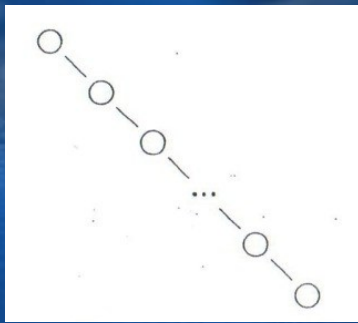


3、【NOIP2003】一个高度为h的二叉树最小元素数目是（ ）。

A.  $2h+1$       B.  $h$       C.  $2h-1$       D.  $2h$       E.  $2^h-1$

【答案】B

【分析】高度为h的最小元素数目的二叉树如下图所示，它有h个节点。

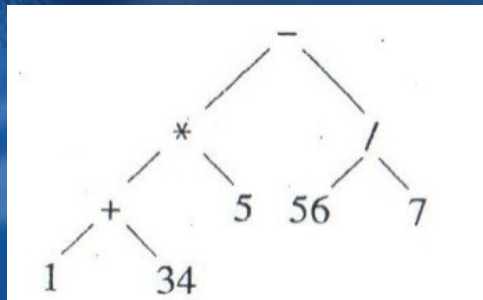


4、【NOIP2003提高组】表达式 $(1+34)*5-56/7$ 的后缀表达式为（ ）。

- A.  $1+34*5-56/7$       B.  $-*+1\ 34\ 5/56\ 7$       C.  $1\ 34\ +5*56\ 7/-$   
D.  $1\ 34\ 5*+56\ 7/-$       E.  $1\ 34+5\ 56\ 7-*/$

【答案】C

【分析】将原式按二叉树展开如图所示，按后根序遍历可得后缀表达式。

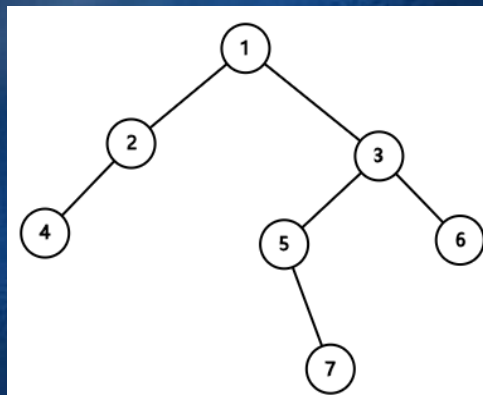


5、【NOIP2004】二叉树T，已知其前序遍历序列为1 2 4 3 5 7 6，中序遍历序列为4 2 1 5 7 3 6，则其后序遍历序列为（ ）。

- A. 4 2 5 7 6 3 1      B. 4 2 7 5 6 3 1      C. 4 2 7 5 3 6 1  
D. 4 7 2 3 5 6 1      E. 4 5 2 6 3 7 1

【答案】B

【分析】根据前序遍历和中序遍历的结果可以画出以下二叉树，按照后续要求读取即可。



6、【NOIP2004】满二叉树的叶结点个数为 $N$ ，则它的结点总数为（ ）。

A.  $N$       B.  $2 * N$       C.  $2 * N - 1$       D.  $2 * N + 1$       E.  $2^N - 1$

【答案】C

【分析1】根据二叉树的性质， $n_0 = n_2 + 1$ ，满二叉树没有度为1的结点，经计算选C。

【分析2】总共是 $2^n - 1$ ，第 $n$ 排叶子总数 $2^{(n-1)}$ ，最后一排是 $n$ ，前面所有的和是 $n-1$ ，答案 $2n-1$ 。

7、【NOIP2005普及组】完全二叉树的结点个数为11，则它的叶结点个数为（ ）。

A. 4      B. 3      C. 5      D. 2      E. 6

【答案】E

【分析】左下角不许缺，右下可以。

8、【NOIP2005普及组】二叉树T的宽度优先遍历序列为A B C D E F G H I，已知A是C的父结点，D是G的父结点，F是I的父结点，树中所有结点的最大深度为3（根结点深度设为0），可知F的父结点是（ ）。

A. 无法确定      B. B      C. C      D. D      E. E

【答案】C

【分析】A|BC|DEF|GHI。



9、【NOIP2005提高组】完全二叉树的结点个数为 $4 * N + 3$ ，则它的叶结点个数为（ ）。

A.  $2 * N$  B.  $2 * N - 1$  C.  $2 * N + 1$  D.  $2 * N - 2$  E.  $2 * N + 2$

【答案】E

【分析】最后一个(叶)结点的编号为 $4*N+3$ ,其父结点的编号为 $2*N+1$ ,因此,  
 $4*N+3-(2*N+1)=2*N+2$

10、【NOIP2006】高度为  $n$  的均衡的二叉树是指：如果去掉叶结点及相应的树枝，它应该是高度为  $n-1$  的满二叉树。在这里，树高等于叶结点的最大深度，根结点的深度为 0，如果某个均衡的二叉树共有 2381 个结点，则该树的树高为（ ）。

A. 10 B. 11 C. 12 D. 13

【答案】B

【分析】 $2^{11}=2048$ 。如果根节点的深度为1，则满二叉树节点总数为 $2^{(深度-1)}$ ，由于 $2048-1=2047<2381$ ，故整个树高为 $11(\text{满二叉树})+1=12$ 。由于本题设定的根节点深度为0，故该题的树高为11。

11、【NOIP2006普及组】已知 6 个结点的二叉树的先根遍历是 1 2 3 4 5 6（数字为结点的编号，以下同），后根遍历是 3 2 5 6 4 1，则该二叉树的可能的中根遍历是（ ）。

- A. 3 2 1 4 6 5      B. 3 2 1 5 4 6  
C. 2 1 3 5 4 6      D. 2 3 1 4 6 5

【答案】B

【分析】先根、中根、后根分别指的是先序、中序、后序，可以根据先根和任一可能的中根形成一棵二叉树，然后读取其后根遍历，如果和题干中后根遍历相同即为答案。

12、【NOIP2007普及组】已知7个结点的二叉树的先根遍历是 1 2 4 5 6 3 7（数字为结点的编号，以下同），中根遍历是 4 2 6 5 1 7 3，则该二叉树的后根遍历是（ ）。

- A. 4 6 5 2 7 3 1    B. 4 6 5 2 1 3 7    C. 4 2 3 1 5 4 7    D. 4 6 5 3 1 7 2

【答案】A

【分析】1为根，2为左子根，3为右子根，6为5的左子，7为3的左子。

谢谢